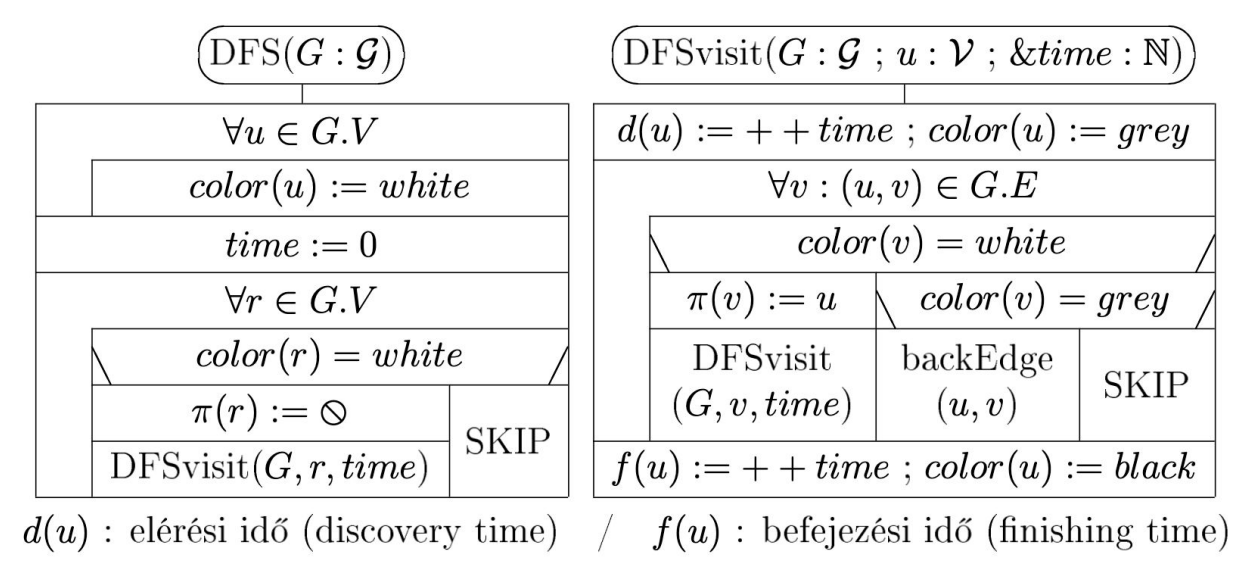
8. Gyakorlat

Algoritmusok és adatszerkezetek 2.

Gráfok mélységi bejárása

Depth First Search / DFS

Mélységi bejárás algoritmusa

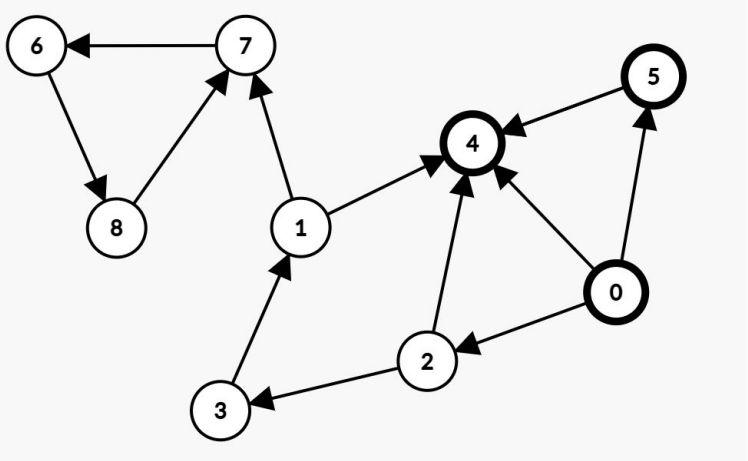


1. Feladat

Játsszuk le az alábbi gráfon a mélységi bejárás algoritmusát a 0 csúcsból indulva.

Adjuk meg, a d, f, Π tömböket és az egyes élekre írjuk rá azok típusát.

A szomszédokat növekvő sorban járjuk be.

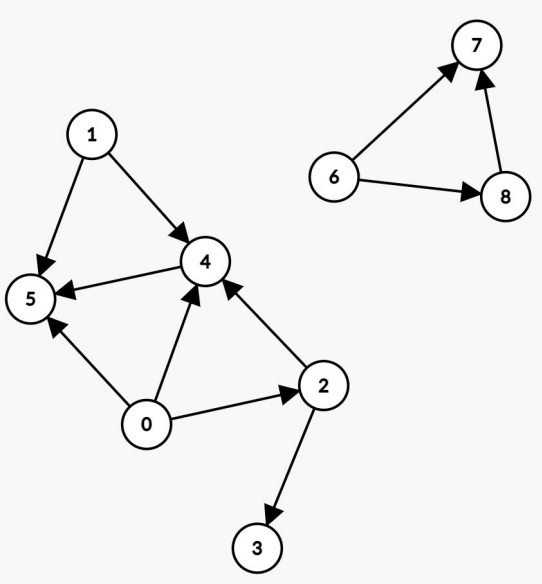


2. Feladat

Játsszuk le az alábbi gráfon a mélységi bejárás algoritmusát a 0 csúcsból indulva.

Adjuk meg, a d, f, Π tömböket és az egyes élekre írjuk rá azok típusát.

A szomszédokat növekvő sorban járjuk be.



Mélységi bejárás alkalmazásai

DAG tulajdonság, topologikus rendezés

Irányított körmentes gráf - DAG

Olyan irányított gráf, amely nem tartalmaz irányított kört.

Ellenőrzése: ha a DFS algoritmus során nem találunk visszaélt.

Topologikus rendezlés

Információlisan: irányított gráf topologikus rendezése alatt a gráf csúcsainak olyan sorba rendezését értjük, amelyben minden él egy-egy később jövő csúcsba (szemléletesen: balról jobbra) mutat.

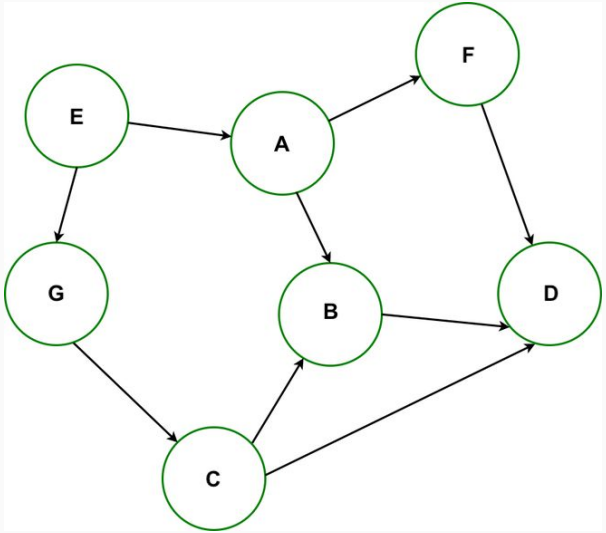
Csak akkor létezik, ha DAG tulajdonságú a gráf

3. Feladat

Adjunk algoritmust, amellyel egy DAG tulajdonságú gráfot topologikusan rendezünk.

Szemléltessük az alábbi gráfon.

Adjuk meg elvi eljárásként majd struktogramban is.



3. Feladat - megoldás

1. Határozzuk meg a csúcsok befokait (hány él vezet beléjük).
2. Kiválasztunk egy csúcsot, aminek a befoka 0, majd rakjuk be a rendezésbe.
    a. Ilyen csúcsot mindig tudunk találni, ha a gráf DAG
3. Töröljük a kiválasztott csúcsot, az éleivel együtt. Csökkentsük ennek alapján a többi csúcs befokát.
4. Térjünk vissza a 2. lépésre, és folytassuk ugyanígy, amíg el nem fogynak a csúcsok.

3. Feladat - Struktogram

Milyen műveletekre van szükség?

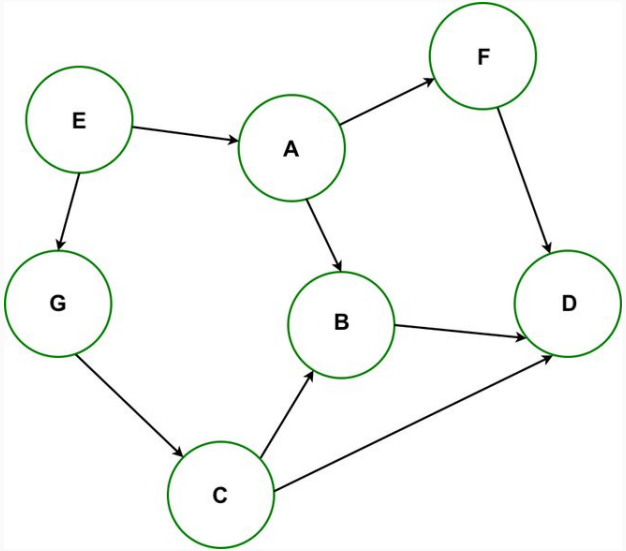
- Befok(v)
- Töröl(v)
- Szomszédok(v)

Topologikus rendezés DFS segítségével

1. Mélységi bejárás hívása G gráfra
2. minden csúcsra meghatározzuk f(v) befejezési időt
3. a csúcsok elhagyásakor szúrjuk be azokat egy láncolt lista elejére
4. return a csúcsok láncolt listája

4. Feladat

Szemléltessük a “DAG topologikus rendezése DFS segítségével“ algoritmust az alábbi gráfon:



Minimális feszítőfák

Tetszőleges feszítőfa

Adjunk algoritmust, amely egy irányítatlan G gráf tetszőleges feszítőfáját meghatározza.

Mimális feszítőfa

Egy irányítatlan G = (V, E) gráf minden (u, v) éle rendelkezik egy w(u, v) súllyal, ami az u és v élek összekötésének költségét adja meg. A cél megtalálni azt a T ⊆ E részhalmazt, aminek segítségével az összes csúcs összekapcsolható, és amelynek a

$$w(T) = \sum_{(u,v)\in T} w(u,v)$$

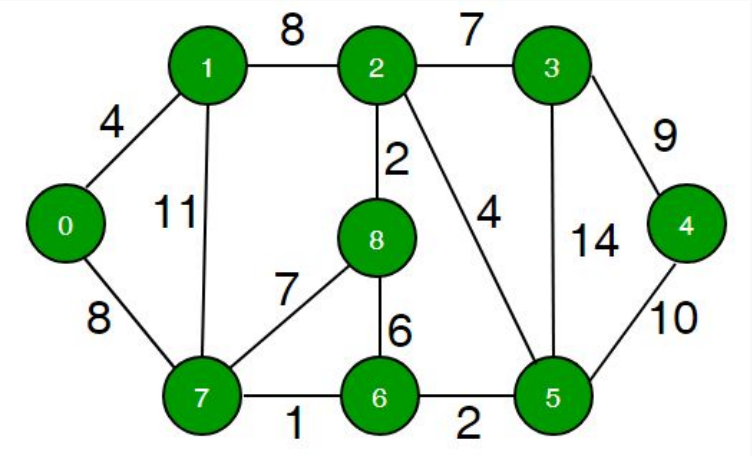
a lehető legkisebb.

Kruskal algoritmus

1. A halmaz üres kezdőértéket kap, és létrejön a gráf csúcsait külön-külön tartalmazó |V| darab fa.
2. Egy ciklussal megyünk, amíg van feldolgozatlan él. Kivesszük a következő legkisebb súlyú élt.
3. Minden így kapott (u, v) élre meg kell vizsgálni, hogy a végpontjaik ugyanahhoz a fához tartoznak-e.
    a. Ha igen, akkor nem kell az éllel foglalkozni (az erdőben kör alakulna ki).
    b. Ha nem, akkor (u, v) élt hozzá kell venni A-hoz, és a két fa csúcsait összevonjuk. (Ebben az esetben a két csúcs különböző fákhoz tartozott.

5. Feladat

Szemléltessük a Kruskal algoritmus működését az alábbi gráfon:



Unió holvan adatszerkezet

Az "unió-holvan" adatszerkezet a következő probléma megoldására született: adott véges sok halmaz, amelyeken a következő lépéseket kell - előre nem ismert sorrendben, sokszor egymás után - elvégezni:

- megkérdezhetjük két elemről, hogy azonos halmazban vannak-e;
- egyesíthetünk két halmazt

Vázlatosan a következőt csináljuk: a halmazokat (felfelé) irányított fákkal reprezentáljuk. Minden elemnek lesz egy "szülő-pointere", a fa gyökerének önmaga. Egy elem halmazát a fája gyökerével azonosítjuk. Az apa-pointerek követésével ez gyorsan meghatározható. Két halmaz egyesítésekor az egyik gyökerének apa-pointerét a másik gyökerére állítjuk.

6. Feladat

Adjuk meg a Kruskal algoritmust, használva az Unió holvan adatszerkezetet!

Absztrakt leírás

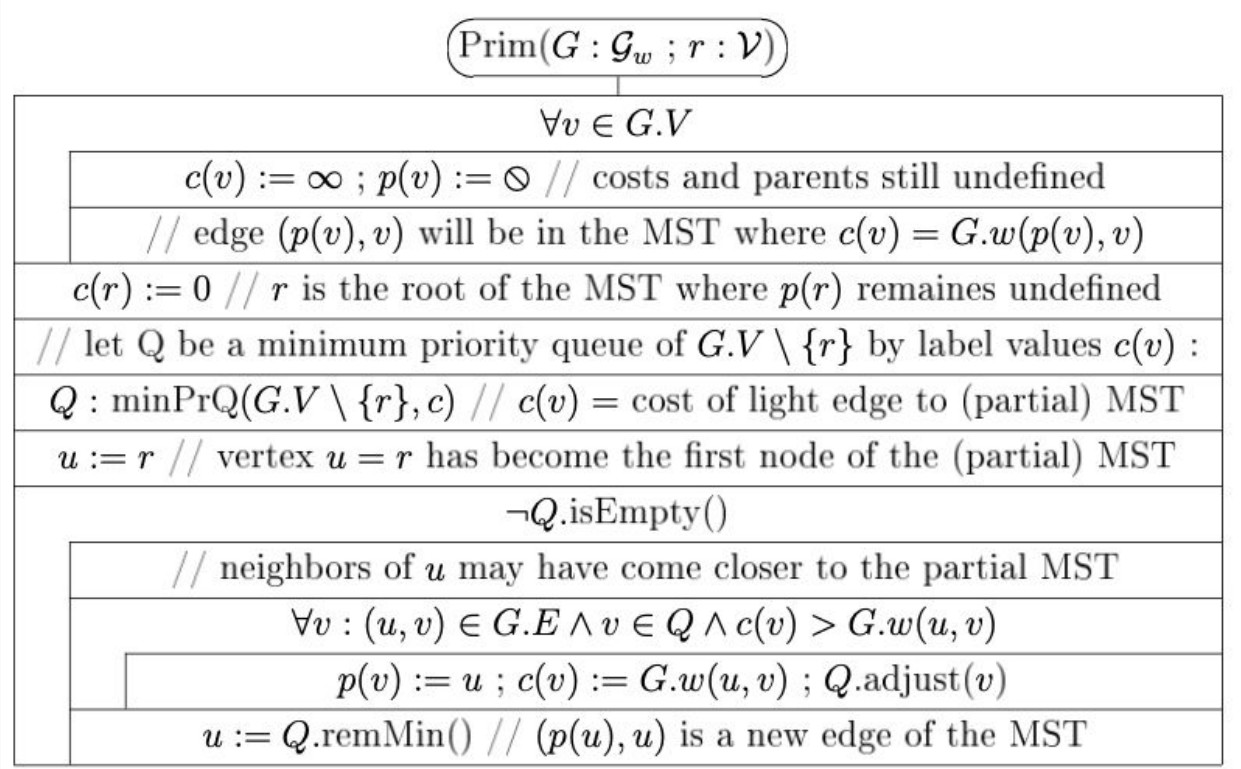
DB := 0; FA := {}
Ciklus az élek élsúly szerint növekvően rendezett listáján amíg DB < N - 1
    EL := a soron következő él
    Ha FA+{EL} nem tartalmaz kört akkor
        FA := FA+{EL}
        DB := DB + 1
    Elágazás vége
Ciklus vége

Prim algoritmusa

Az A halmaz egyetlen fa. Az A-hoz hozzáadott biztonságos él mindig a legkisebb súlyú olyan él, amelyik egy A-beli és egy A-n kívüli csúcsot köt össze. Mivel mohó algoritmus, így minden lépésben olyan éllel bővíti a fát, amely a lehető legkisebb mértékben növeli a fa teljes súlyát.

Minden lépésben azt a könnyű élt vesszük hozzá az A fához, amelyik egy A-beli csúcsot köt össze a GA = (V, E) egy izolált csúcsával.

Prim algoritmusa



6. Feladat

Szemléltessük a Prim algoritmus működését az alábbi gráfon:

